08
PHASE

AI-Era Analytics

Augment Intelligence. Automate Insights.
Predict. Personalize. Transform.

- AI/ML Models
- Natural Language Processing
- Intelligent Automation
- Predictive Analytics
- Personalized Insights

- Leverage AI Models
- Automate Workflows
- Discover Deeper Insights
- Make Smarter Decisions
- Drive Business Transformation

The Future of Analytics is AI-Powered. Are You Ready?

Unit 1: Advanced Prompt Engineering for Analytics

1.1 Analytical Persona	1.2 Chain-of- Thought	1.3 Few-Shot Learning	1.4 Delimiters & Output
1.5 Iterative Debugging			

1.1 The Analytical Persona

Crafting Prompts That Force the AI to Act as a Senior Data Scientist

DEFINITION

The Analytical Persona is a prompt engineering technique where you instruct an AI to behave as a specific expert role — in this case, a Senior Data Scientist — before asking it a question. By setting the 'character' or 'mindset' of the AI upfront, you dramatically improve the quality, depth, and relevance of its responses.

Think of it as giving the AI a job title and a set of responsibilities before you ask it to do any work.

SUB-TOPICS

- **Role Definition: Explicitly state who the AI should act as.**

Describe the experience level, industry context, and priorities of that persona.

- **Persona Attributes: Define the skills and mindset of the role.**

E.g., rigorous with data quality, prefers efficient SQL, explains assumptions.

- **Context Setting: Provide the business environment.**

Tell the AI what company it works for, what tools are used, what the goal is.

- **Constraint Adding: Set rules the persona must follow.**

E.g., 'Never use SELECT *', 'Always validate data types', 'Cite your assumptions'.

WHY IT IS IMPORTANT

- Generic prompts produce generic answers. A persona focuses the AI on domain-specific logic.
- It reduces the chance of the AI giving overly simplified, incorrect, or irrelevant answers.
- For data analysts, this means getting SQL queries, Python scripts, and explanations that actually match professional standards.
- It saves time — fewer revisions, better first drafts.

REAL-WORLD ANALOGY

Imagine you walk into a hospital and ask a random person, 'What's wrong with me?' You'd get a vague answer. But if you ask a Senior Cardiologist with 20 years of experience to evaluate your symptoms — using real test

data — you get a precise, professional diagnosis. The Analytical Persona works the same way: you're not asking 'just anyone' — you're asking a highly defined expert.

SYNTAX / PROMPT STRUCTURE

```
You are a [ROLE] with [X years/level] of experience in [DOMAIN].
You are working at [COMPANY TYPE] that uses [TOOLS/DATABASES].
Your goal is to [PRIMARY OBJECTIVE].
You always [RULE 1] and [RULE 2].
You never [RESTRICTION 1].

Now, answer the following question:
[YOUR ACTUAL QUESTION]
```

EXAMPLE

Prompt Without Persona:

```
Write a SQL query to find top customers.
```

Prompt With Analytical Persona:

```
You are a Senior Data Scientist with 10 years of experience in e-commerce
analytics.
You work at a B2C retail company that uses PostgreSQL and dbt.
Your goal is to help the marketing team identify high-value customers.
You always write clean, well-commented SQL and explain your logic.
You never use SELECT * and always include data type checks.

Write a SQL query to find the top 10 customers by total revenue
in the last 90 days, excluding refunded orders.
```

Result: The AI now gives a well-structured, production-ready SQL query with comments, proper filtering, and an explanation — far better than without the persona.

INDUSTRY USE CASE

Industry	Use Case

E-Commerce	Persona as a Marketing Analyst to build customer segmentation queries
Healthcare	Persona as a Clinical Data Analyst to analyze patient outcome metrics
Finance	Persona as a Risk Analyst to detect anomalous transactions in SQL
Retail	Persona as a Supply Chain Analyst to forecast inventory shortfalls

1.2 Chain-of-Thought (CoT) Prompting

Forcing LLMs to Explain Their Logic Step-by-Step to Reduce Math Errors

📖 DEFINITION

Chain-of-Thought (CoT) Prompting is a technique where you instruct an AI to think out loud — showing each step of its reasoning before giving a final answer. Instead of jumping straight to a result, the AI walks through the logic, calculations, or decisions one by one.

This is especially important for analytical tasks involving math, multi-step logic, or complex SQL — where a single skipped step can lead to a completely wrong answer.

🔍 SUB-TOPICS

- **Zero-Shot CoT: Add 'Let's think step by step' to any prompt.**

The simplest form — just one instruction triggers the chain-of-thought behavior.

- **Manual CoT: You write out the steps you want the AI to follow.**

More control — you define the exact reasoning path.

- **Self-Consistency CoT: Ask the AI to generate multiple reasoning paths and pick the most common answer.**

Useful for reducing hallucination in complex calculations.

- **Structured CoT: Ask the AI to use specific headers like Step 1, Step 2, Final Answer.**

Makes the output easier to review and debug.

⭐ WHY IT IS IMPORTANT

- LLMs (Large Language Models) are trained to predict the next word, not to do real math. Without CoT, they often hallucinate numeric results.

- CoT forces the AI to slow down and show its work — like a student writing out exam steps.
- For data analysts, this is critical: a wrong JOIN logic or a bad aggregation formula can corrupt entire dashboards.
- It also makes the AI's output auditable — you can check where it went wrong.

REAL-WORLD ANALOGY

Imagine asking a junior accountant to calculate tax liability. If they just hand you a final number — you can't trust it. But if they show: Step 1: Revenue = $500,000 | Step 2: Deductions = $80,000 | Step 3: Taxable Income = $420,000 | Step 4: Tax Rate = 30% | Final Tax = $126,000 — now you can review each step and catch mistakes. CoT prompting does exactly this for AI.

SYNTAX / PROMPT STRUCTURE

```
// Zero-Shot CoT (simplest)
[Your question here]. Let's think step by step.

// Structured CoT
Answer the following question by breaking it into clear steps.
Format your answer as:
 Step 1: [What you are doing]
 Step 2: [Next action]
 ...continue until complete...
 Final Answer: [Result]

Question: [YOUR QUESTION]
```


EXAMPLE

Prompt Without CoT:

```
What is the Month-over-Month revenue growth rate if January
revenue was $120,000 and February revenue was $145,000?
```

AI Answer (bad): ~20% growth ← might be wrong or unexplained.

Prompt With CoT:

```
What is the Month-over-Month revenue growth rate if January
```

```
revenue was $120,000 and February revenue was $145,000?
Let's think step by step.

// AI Output:
Step 1: Identify the values.
 January Revenue = $120,000
 February Revenue = $145,000
Step 2: Calculate the difference.
 Difference = $145,000 - $120,000 = $25,000
Step 3: Divide difference by the original value.
 $25,000 / $120,000 = 0.2083
Step 4: Convert to percentage.
 0.2083 × 100 = 20.83%
Final Answer: Month-over-Month revenue growth = 20.83%
```

INDUSTRY USE CASE

Industry	Use Case
Finance	Step-by-step calculation of compound interest, loan amortization, or IRR
E-Commerce	CoT for building complex funnel conversion rate calculations
Healthcare	Step-by-step statistical analysis of clinical trial outcome data
Manufacturing	Breaking down defect rate formulas: defects/total units × 100

1.3 Few-Shot Learning

Providing Examples of Your Schema and 'Gold Standard' SQL to Improve AI Output

DEFINITION

Few-Shot Learning is a prompting technique where you give the AI a small number of examples (usually 2 to 5) of exactly what you want — before asking it to do the actual task. These examples act as a template, teaching the AI your preferred style, format, and logic without any formal training.

In data analytics, this means showing the AI your database schema (table and column names) and a few 'Gold Standard' SQL queries — queries you've already written and verified — so the AI can follow the same pattern for new questions.

SUB-TOPICS

- **Zero-Shot Learning: No examples given — AI guesses from scratch.**

High risk of hallucinated column names and wrong table references.

- **One-Shot Learning: One example provided.**

Better than zero-shot, but still limited in pattern guidance.

- **Few-Shot Learning: 2 to 5 examples provided.**

The sweet spot — enough examples for the AI to learn the pattern reliably.

- **Schema Injection: Providing CREATE TABLE statements or column lists.**

This tells the AI exactly what columns and data types exist so it stops inventing them.

- **Gold Standard Examples: Pre-verified, high-quality queries you share as examples.**

These show the AI your coding style, naming conventions, and logic preferences.

WHY IT IS IMPORTANT

- Without schema context, AI often invents column names that don't exist — a phenomenon called hallucination.
- Few-Shot examples dramatically reduce these errors by anchoring the AI to your real data structure.
- It enforces consistency — all AI-generated queries match your team's coding standards.
- Faster onboarding: New analysts can use AI as a tutor by feeding it the company's query library.

REAL-WORLD ANALOGY

Imagine hiring a new chef and handing them a recipe book with 3 of your restaurant's best dishes. You say: 'All our new dishes must follow this style — same ingredients format, same plating instructions, same portion sizes.' The chef now has a template. They don't guess your style; they replicate it. Few-Shot Learning is your recipe book for the AI.

SYNTAX / PROMPT STRUCTURE

```
Here is the schema for our database:
```

```
 Table: orders (order_id INT, customer_id INT, order_date DATE,
 revenue FLOAT, status VARCHAR)
 Table: customers (customer_id INT, name VARCHAR, region VARCHAR, signup_date
DATE)

Here are two example queries (Gold Standard):

Example 1 - Find monthly revenue:
 SELECT DATE_TRUNC('month', order_date) AS month,
 SUM(revenue) AS total_revenue
 FROM orders
 WHERE status = 'completed'
 GROUP BY 1 ORDER BY 1;

Example 2 - Count customers by region:
 SELECT region, COUNT(DISTINCT customer_id) AS customer_count
 FROM customers
 GROUP BY region ORDER BY customer_count DESC;

Now write a query to: [YOUR NEW QUESTION]
```

EXAMPLE

New Question (with Few-Shot context):

```
Now write a query to find the top 5 regions by total revenue
from completed orders in Q1 2024.

// AI Output (using schema correctly):
SELECT c.region,
 SUM(o.revenue) AS total_revenue
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status = 'completed'
 AND o.order_date BETWEEN '2024-01-01' AND '2024-03-31'
GROUP BY c.region
ORDER BY total_revenue DESC
LIMIT 5;
```

The AI correctly used the real column names, the correct JOIN logic, and the right filter — all learned from the examples.

INDUSTRY USE CASE

Industry	Use Case
Banking	Provide transaction schema + sample fraud detection queries as examples
SaaS	Feed product event schema + churn queries so AI generates cohort analysis
Retail	Share inventory schema + reorder queries to generate stock forecasting SQL
HR Analytics	Provide employee table schema + examples to generate attrition queries

1.4 Delimiters & Structured Output

Getting AI to Return Data in Specific Formats (JSON, Markdown, Python Dictionaries)

DEFINITION

Delimiters are special characters or markers (like triple backticks ```, XML tags <>, or dashes ---) that you use to clearly separate different parts of a prompt — instructions from data, context from questions. Structured Output means explicitly telling the AI to return its response in a specific, machine-readable format such as JSON, Markdown tables, Python dictionaries, or CSV.

Together, these two techniques make AI output reliable enough to use directly in code, dashboards, or further automated processing — without manually reformatting the response.

SUB-TOPICS

- **Delimiters for Prompt Clarity:**
  - Triple backticks (```) — Separate code blocks or raw data sections
  - XML-style tags (<data>, <question>) — Label different sections clearly
  - Triple dashes (---) — Mark beginning and end of instructions
  - Hash symbols (###) — Denote instruction sections
- **Structured Output Formats:**
  - JSON — Best for API responses and downstream code consumption

- Markdown Table — Best for human-readable reports and dashboards
- Python Dictionary — Best for feeding data directly into Python scripts
- CSV — Best for tabular data export and import into Excel or BI tools

- **Output Schema Definition: Explicitly define what fields you want in the output.**

Tell the AI exactly what keys, columns, or headers to include.

WHY IT IS IMPORTANT

- Without structure, AI gives verbose, unstructured text that's hard to use programmatically.
- JSON and dict outputs can be directly fed into Python pipelines, APIs, or databases.
- Delimiters prevent the AI from confusing your data with its instructions — reducing errors.
- Enables automation: AI output becomes part of an automated data pipeline, not just a chat message.

REAL-WORLD ANALOGY

Imagine calling a supplier and saying 'Send me product details.' You'd get an email in any random format. But if you say 'Fill in this form: Product Name | SKU | Price | Stock Level' — you get structured, usable data every time. Delimiters and Structured Output are your form template for the AI.

SYNTAX / PROMPT STRUCTURE

```
// Using Delimiters to separate context from question
###CONTEXT###
The following is a list of sales transactions:
```
customer_id, product, amount, date
101, Laptop, 1200, 2024-01-05
102, Phone, 800, 2024-01-06
```

###QUESTION###
Summarize the above data and return your answer ONLY as a JSON object
with these keys: total_transactions, total_revenue, average_order_value

// Expected AI Output:
{
```

```
 "total_transactions": 2,
 "total_revenue": 2000,
 "average_order_value": 1000
}
```

EXAMPLE — Multiple Output Formats

Requesting a Markdown Table:

```
Prompt: List the top 3 products by revenue.
Return your answer as a Markdown table with columns:
Rank | Product | Revenue

AI Output:
| Rank | Product | Revenue |
|------|---------|---------|
| 1 | Laptop | $45,000 |
| 2 | Phone | $32,000 |
| 3 | Tablet | $21,000 |
```

Requesting a Python Dictionary:

```
Prompt: Give the same data as a Python dictionary.

AI Output:
top_products = {
 'Laptop': 45000,
 'Phone': 32000,
 'Tablet': 21000
}
```

INDUSTRY USE CASE

Industry	Use Case
Tech / SaaS	AI returns JSON-formatted KPI summaries consumed directly by a React dashboard
Finance	Structured CSV output from AI fed directly into Excel financial models

Marketing	AI returns Markdown table of campaign metrics for weekly stakeholder reports
Data Engineering	AI generates Python dicts of transformation rules for ETL pipeline configs

1.5 Iterative Debugging

How to Talk Back to an AI When It Gives You a Hallucinated Column Name

📖 DEFINITION

Iterative Debugging is the process of going back and forth with an AI — providing feedback, corrections, and additional context — to progressively improve its output until you get a correct and usable result. Just like debugging code, you identify the error, tell the AI what went wrong, and ask it to fix only that part.

In data analytics, hallucinations commonly appear as: column names that don't exist, wrong table references, incorrect aggregation logic, or made-up function names. Iterative Debugging is the structured method to fix these without starting over.

🔍 SUB-TOPICS

- **Hallucination Identification: Recognizing when the AI has invented something.**

Signs: column names not in your schema, syntax errors, functions that don't exist in your SQL dialect.

- **Targeted Correction: Pointing to the specific error rather than rewriting the whole prompt.**

More efficient — only fix what's broken, preserve what's correct.

- **Schema Reinforcement: Re-injecting your schema in the correction message.**

Remind the AI of the correct column names when it hallucinates.

- **Constraint Adding: Adding new rules during the debug conversation.**

E.g., 'Do not use window functions', 'Use only these 3 tables'.

- **Error Explanation: Sharing the actual error message from your database.**

Feed the SQL error directly to the AI so it can diagnose and fix precisely.

- **Verification Loop: After each fix, test in your database and report back.**

Repeat until the query runs correctly and gives the right result.

WHY IT IS IMPORTANT

- AI is not perfect. Even with great prompts, hallucinations happen — especially with complex schemas.
- Iterative Debugging saves time: fixing incrementally is faster than rewriting everything from scratch.
- It trains you to be a better prompt engineer — each debugging session reveals what context was missing.
- In production analytics, systematic debugging ensures queries are correct before they touch real data.

REAL-WORLD ANALOGY

Imagine working with a new software developer who writes code for your project. They make a mistake — wrong variable name. You don't fire them and hire someone else. You say: 'The variable is called customer_id, not client_id. Here is the schema — please fix just that part.' That conversation IS iterative debugging. You correct, clarify, and confirm — until it works.

ITERATIVE DEBUGGING WORKFLOW

```
STEP 1 — Initial Prompt:
 [Ask AI to write a query]

STEP 2 — Test in Database:
 Run the query → Get error or wrong result

STEP 3 — Error Feedback Prompt:
 'The query you wrote has an error: [PASTE ERROR MESSAGE].
 The column you used ([wrong_column]) does not exist.
 The correct column name is [correct_column].
 Here is the actual schema: [PASTE SCHEMA].
 Please fix only the column reference and re-output the full corrected query.'

STEP 4 — Re-test:
 Run the fixed query → Check result

STEP 5 — Logic Correction (if needed):
```

```
 'The query runs but the result is wrong. It should show [X] but shows [Y].
 The issue is likely [SUSPECTED CAUSE]. Please revise the [specific clause].'

STEP 6 — Confirm & Finalize:
 Verify output matches expected business result.
```

EXAMPLE

Round 1 — AI Hallucinates:

```
AI Output:
 SELECT client_name, SUM(sale_amount)
 FROM sales_data
 GROUP BY client_name

Error from DB: Column 'client_name' does not exist.
Error from DB: Table 'sales_data' does not exist.
```

Round 2 — Your Correction Prompt:

```
'You used wrong names. Here is the real schema:
 Table: orders (order_id, customer_id, revenue, order_date, status)
 Table: customers (customer_id, full_name, region)
The column for customer name is full_name in the customers table.
The revenue column is called revenue, not sale_amount.
Please rewrite the query using only these exact names.'
```

Round 3 — Corrected AI Output:

```
SELECT c.full_name, SUM(o.revenue) AS total_revenue
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status = 'completed'
GROUP BY c.full_name
ORDER BY total_revenue DESC;
```

INDUSTRY USE CASE

Industry	Use Case

E-Commerce	Debug AI-generated funnel queries by feeding Redshift error messages back to AI
Finance	Iteratively fix AI-written risk scoring SQL until it matches audit expectations
Healthcare	Correct hallucinated ICD code column names in patient analysis queries
Logistics	Fix wrong JOIN conditions in AI-generated shipment tracking queries

Quick Reference Summary — Unit 1

#	Technique	Core Idea	Key Benefit	When to Use
1.1	**Analytical Persona**	Define AI role before asking questions	Domain-specific, professional responses	Every analytics prompt
1.2	**Chain-of-Thought**	Ask AI to show step-by-step reasoning	Fewer math & logic errors	Calculations & multi-step queries
1.3	**Few-Shot Learning**	Provide schema + example Gold Standard queries	No hallucinated column names	Complex or new schema work
1.4	**Delimiters & Output**	Separate prompt sections & specify output format	Machine-ready, structured output	Automation & pipelines
1.5	**Iterative Debugging**	Feed errors back to AI and correct incrementally	Fast, precise error resolution	When AI output has errors

UNIT 1

Advanced Prompt Engineering for Analytics

Interview-Base Unit Test

Covers: Analytical Persona • Chain-of-Thought • Few-Shot Learning
Delimiters & Structured Output • Iterative Debugging

Level	Description	Question Count
Level 1	Warmup Test	5 MCQ
Level 2	Interviewer Test	4 Descriptive + 4 Scenario + 4 Coding + 3 DB Coding

LEVEL 1 — Warmup Test

5 Multiple Choice Questions | Circle / mark the correct option

Q1. What is the primary purpose of the Analytical Persona technique in prompt engineering?

(A) To make AI responses shorter and more concise	(B) To instruct the AI to behave as a specific expert role before asking a question
(C) To provide multiple examples of SQL queries to the AI	(D) To separate different sections of a prompt using special characters

Answer

Q2. Which Chain-of-Thought variant requires the least effort from the user to activate step-by-step reasoning?

(A) Manual CoT — you write out every step the AI must follow	(B) Self-Consistency CoT — ask AI to generate multiple reasoning paths
(C) Zero-Shot CoT — append "Let's think step by step" to the prompt	(D) Structured CoT — use Step 1, Step 2, Final Answer headers

☑ **Answer**

Q3. In Few-Shot Learning, what is a "Gold Standard" example?

(A) An example query written by the AI itself during a previous session	(B) A query that uses SELECT * for maximum data retrieval
(C) A pre-verified, high-quality query that demonstrates the preferred style and logic	(D) A schema diagram exported directly from the database

☑ **Answer**

Q4. Which output format is BEST suited when AI-generated data must be consumed directly by a Python pipeline?

(A) Plain paragraph text	(B) Markdown table
(C) JSON or Python dictionary	(D) CSV exported to Excel

☑ **Answer**

Q5. During Iterative Debugging, what is the most efficient strategy when an AI hallucinates a column name?

(A) Discard the conversation and start a completely new prompt from scratch	(B) Accept the wrong column name and manually fix the query yourself
(C) Point to the specific wrong column, re-inject the correct schema, and ask the AI to fix only that part	(D) Add more examples from unrelated databases to confuse the AI less

☑ **Answer**

Section A — Descriptive Questions

Q1 [Descriptive]

Explain the concept of Analytical Persona in prompt engineering. Why does simply asking an AI a question without a persona often lead to suboptimal results for data analytics tasks?

Answer

Q2 [Descriptive]

Describe the four main variants of Chain-of-Thought (CoT) prompting. When would you prefer Structured CoT over Zero-Shot CoT in a data analytics context?

Answer

Q3 [Descriptive]

What is Schema Injection in Few-Shot Learning? How does it prevent the problem of AI hallucinating column names? Provide a structured explanation with an example.

Answer

Q4 [Descriptive]

Explain the role of Delimiters in prompt engineering. Compare at least three delimiter types and describe the type of output format you would request for each of the following use cases: (a) feeding data to a React dashboard, (b) exporting to Excel, (c) embedding in a weekly stakeholder report.

Answer

Section B — Scenario-Based Descriptive Questions

Q5 [Scenario-Based]

Scenario: You are a junior analyst at a healthcare company. Your manager asks you to use AI to analyse patient readmission rates. The AI keeps giving generic statistical responses that do not align with your clinical database structure or the team's goals. What specific Analytical Persona prompt would you design? Write out the full persona prompt and explain each component you included.

☑ **Answer**

Q6 [Scenario-Based]

Scenario: You ask an AI to calculate the compound annual growth rate (CAGR) for your company's revenue over 5 years. The AI immediately gives you a final number — 14.2% — with no explanation. Your finance manager says the number seems off. How would you redesign the prompt using Chain-of-Thought techniques to get a verifiable, step-by-step answer? Write the improved prompt and explain why each addition helps.

☑ **Answer**

Q7 [Scenario-Based]

Scenario: You are onboarding a new analyst to your e-commerce team. The team uses a complex PostgreSQL database with 12 tables. The new analyst wants to use AI to help write queries, but previous AI attempts produced SQL with columns that don't exist. Design a Few-Shot Learning prompt template for the new analyst to use. Include schema injection, two Gold Standard examples, and explain what makes each example "gold standard."

☑ **Answer**

Q8 [Scenario-Based]

Scenario: You run an AI-generated SQL query in Redshift and receive this error: "column "sale_amount" does not exist." You also notice the result should show only Q3 2024 data, but no date filter was applied. Using the Iterative Debugging framework, write out your complete Round 2 correction prompt. Explain which debugging sub-technique you are applying at each step.

Answer

Section C — Coding Questions

Q9 [Coding]

Write an Analytical Persona prompt in Python that uses the OpenAI / Anthropic API to query a data science assistant. The persona should be a Senior Data Scientist working in fintech, restricted from using SELECT *, and asked to find the top 5 customers by transaction volume in the last 30 days.

Answer

Q10 [Coding]

Write a Python function called cot_prompt_builder() that accepts a math question and automatically wraps it with a Structured Chain-of-Thought template. The function should return the formatted prompt string, and include a sample call demonstrating its use for a Month-over-Month growth rate calculation.

Answer

Q11 [Coding]

Write a Python function called few_shot_prompt_builder() that accepts: (1) a schema dictionary, (2) a list of (question, SQL) tuples as examples, and (3) a new question. It should return a complete Few-Shot prompt string ready to send to an AI model.

Answer

Q12 [Coding]

Write a Python function called iterative_debug_prompt() that takes: (1) the original broken query, (2) the database error message, (3) a correction dictionary {wrong_name: correct_name}, and (4) any additional constraints. It should return a structured Round 2 correction prompt for the AI.

Answer

Section D — Database Scenario-Based Coding (SQL)

Q13 [DB Scenario + SQL]

Scenario: You are a data analyst at a SaaS company. Using the schema below, write an AI-ready Few-Shot prompt that includes the schema, one Gold Standard example, and then solve the new question yourself with a complete SQL query. Schema: Table: subscriptions (sub_id INT, customer_id INT, plan VARCHAR, start_date DATE, end_date DATE, monthly_fee FLOAT, status VARCHAR) Table: customers (customer_id INT, full_name VARCHAR, country VARCHAR, signup_date DATE) New Question: Find the total Monthly Recurring Revenue (MRR) by plan type for all active subscriptions.

Answer

Q14 [DB Scenario + SQL]

Scenario: You are debugging an AI-generated query at a retail company. The AI produced the following broken query: SELECT p.product_name, SUM(o.qty) AS total_sold FROM order_items o JOIN product_catalog p ON o.prod_id = p.product_id WHERE o.sale_date >= CURRENT_DATE - 90 GROUP BY p.product_name ORDER BY total_sold DESC LIMIT 10; But the actual schema is: Table: order_items (item_id INT, order_id INT, product_id INT, quantity INT, item_date DATE) Table: products (product_id INT, name VARCHAR, category VARCHAR, price FLOAT) Identify ALL hallucinations in the query and write: (a) the correction prompt, (b) the fully corrected SQL.

☑ **Answer**

Q15 [DB Scenario + SQL]

Scenario: You are a data engineer at an e-commerce company. Your manager wants a weekly report showing: (1) total revenue by region, (2) average order value by region, and (3) the percentage each region contributes to overall revenue — all for completed orders in the current year. Using the Delimiters & Structured Output technique, write: (a) the full prompt with delimiters requesting JSON output, and (b) the equivalent SQL query that produces this data.

☑ **Answer**

UNIT 1

Advanced Prompt Engineering for Analytics

Interview-Base Unit Test

Covers: Analytical Persona • Chain-of-Thought • Few-Shot Learning
Delimiters & Structured Output • Iterative Debugging

Level	Description	Question Count
Level 1	Warmup Test	5 MCQ
Level 2	Interviewer Test	4 Descriptive + 4 Scenario + 4 Coding + 3 DB Coding

LEVEL 1 — Warmup Test

5 Multiple Choice Questions | Circle / mark the correct option

Q1. What is the primary purpose of the Analytical Persona technique in prompt engineering?

(A) To make AI responses shorter and more concise	**(B) To instruct the AI to behave as a specific expert role before asking a question ✓**
(C) To provide multiple examples of SQL queries to the AI	**(D)** To separate different sections of a prompt using special characters

Explanation: Analytical Persona assigns a defined expert identity (e.g., Senior Data Scientist) to the AI before a question is posed, steering it toward domain-specific, professional responses — rather than generic answers.

Q2. Which Chain-of-Thought variant requires the least effort from the user to activate step-by-step reasoning?

(A) Manual CoT — you write out every step the AI must follow	**(B)** Self-Consistency CoT — ask AI to generate multiple reasoning paths
(C) Zero-Shot CoT — append "Let's think step by step" to the prompt ✓	**(D)** Structured CoT — use Step 1, Step 2, Final Answer headers

Explanation: "Let's think step by step" is a single phrase that triggers Zero-Shot CoT. No examples or custom step definitions are needed — making it the lowest-effort CoT approach.

Q3. In Few-Shot Learning, what is a "Gold Standard" example?

(A) An example query written by the AI itself during a previous session	**(B)** A query that uses SELECT * for maximum data retrieval
(C) A pre-verified, high-quality query that demonstrates the preferred style and logic ✓	**(D)** A schema diagram exported directly from the database

Explanation: Gold Standard examples are queries already written, tested, and validated by the analyst. They show the AI the exact coding style, naming conventions, and logic it should follow for new questions.

Q4. Which output format is BEST suited when AI-generated data must be consumed directly by a Python pipeline?

(A) Plain paragraph text	**(B)** Markdown table
(C) JSON or Python dictionary ✓	**(D)** CSV exported to Excel

Explanation: JSON and Python dictionaries are natively parseable by Python code. They can be ingested directly into scripts, APIs, or databases without manual reformatting — making them ideal for automated pipelines.

Q5. During Iterative Debugging, what is the most efficient strategy when an AI hallucinates a column name?

(A) Discard the conversation and start a completely new prompt from scratch	**(B)** Accept the wrong column name and manually fix the query yourself
(C) Point to the specific wrong column, re-inject the correct schema, and ask the AI to fix only that part ✓	**(D)** Add more examples from unrelated databases to confuse the AI less

Explanation: Targeted correction is the core principle of Iterative Debugging. Re-injecting the schema and pinpointing the exact error is faster and more precise than rewriting everything — preserving what is already correct.

Section A — Descriptive Questions

Q1 [Descriptive]

Explain the concept of Analytical Persona in prompt engineering. Why does simply asking an AI a question without a persona often lead to suboptimal results for data analytics tasks?

Answer

When I first learned about Analytical Persona, I realised it's essentially giving the AI a "job role" before asking it to work. Without a persona, the AI responds as a generalist — it doesn't know whether you want a beginner-friendly explanation or a production-ready SQL query.

An Analytical Persona prompt explicitly states:

- WHO the AI is (e.g., Senior Data Scientist with 10 years in e-commerce)
- WHERE it works (B2C retail company using PostgreSQL)
- WHAT its goal is (help the marketing team identify high-value customers)
- Rules it must follow (never use SELECT *, always add comments)

Why this matters: Generic prompts produce generic answers. The persona narrows the AI's response space to domain-specific logic, professional SQL standards, and relevant assumptions — reducing the need for revisions and producing better first drafts.

Real-world analogy: Asking a random person "What's wrong with me?" versus asking a senior cardiologist with your test results. The persona transforms the quality of the diagnosis.

Q2 [Descriptive]

Describe the four main variants of Chain-of-Thought (CoT) prompting. When would you prefer Structured CoT over Zero-Shot CoT in a data analytics context?

Answer

CoT prompting forces the AI to "show its work" — much like a student showing steps in an exam. There are four main variants:

1. Zero-Shot CoT — Simply append "Let's think step by step." to any prompt. Easiest to use; best for quick calculations.

2. Manual CoT — You define the exact steps for the AI to follow. More control; used when the reasoning path is well-known.

3. Self-Consistency CoT — Ask the AI to generate multiple reasoning paths, then select the most common answer. Reduces hallucinations in complex math.

4. Structured CoT — Ask AI to use labelled headers: Step 1, Step 2 ... Final Answer. Makes output auditable and easy to review.

When to prefer Structured CoT:

In analytics, I would use Structured CoT when the output needs to be reviewed by a stakeholder or audited (e.g., financial calculations, KPI derivations). The labelled steps make it easy to spot exactly where logic went wrong — which is critical when a dashboard formula is feeding decision-making at a senior level.

Zero-Shot CoT is fine for quick ad-hoc math, but when the stakes are higher (audit-ready reports, client-facing metrics), Structured CoT provides traceability.

Q3 [Descriptive]

What is Schema Injection in Few-Shot Learning? How does it prevent the problem of AI hallucinating column names? Provide a structured explanation with an example.

Answer

Hallucination in SQL prompting is when the AI invents table names, column names, or functions that do not exist in your actual database. Schema Injection directly solves this.

Schema Injection means including your CREATE TABLE statements or column lists directly inside the prompt — so the AI knows exactly what exists before it writes a single line of SQL.

Why it works: LLMs learn from training data that includes thousands of database schemas. Without your schema, the AI guesses based on common patterns — and often guesses wrong. When you inject the real schema, the AI is "anchored" to your actual structure.

Example structure:

Table: orders (order_id INT, customer_id INT, order_date DATE, revenue FLOAT, status VARCHAR)

Table: customers (customer_id INT, name VARCHAR, region VARCHAR, signup_date DATE)

With this injected, the AI will use "revenue" instead of inventing "sale_amount", and "full_name" instead of guessing "client_name" — exactly the names that exist in your system.

Combined with Gold Standard query examples (Few-Shot), Schema Injection gives the AI both the vocabulary (columns) and the grammar (coding style) it needs to write accurate queries.

Q4 [Descriptive]

Explain the role of Delimiters in prompt engineering. Compare at least three delimiter types and describe the type of output format you would request for each of the following use cases: (a) feeding data to a React dashboard, (b) exporting to Excel, (c) embedding in a weekly stakeholder report.

Answer

Delimiters are special markers that clearly separate different sections of a prompt — preventing the AI from confusing your data with its instructions.

Three common delimiter types:

• Triple backticks (```) — Used to wrap raw data blocks or code sections. Signals "this is data, not instructions."

• XML-style tags (<data>, <question>) — Clearly label distinct sections. Best when a prompt has multiple distinct parts.

• Hash symbols (###CONTEXT###, ###QUESTION###) — Human-readable section markers; great for long prompts with clear phases.

Output format by use case:

(a) React Dashboard → JSON

JSON is natively parseable by JavaScript. A React component can directly call JSON.parse() on the AI output and render KPIs without transformation.

(b) Excel Export → CSV

CSV can be directly imported into Excel or Power BI. Each row maps to a spreadsheet row — no additional formatting needed.

(c) Weekly Stakeholder Report → Markdown Table

Markdown tables render cleanly in Notion, Confluence, or email-friendly HTML. Stakeholders get a human-readable grid — clear and professional.

The key insight: structured output transforms AI from a conversational tool into a programmable data source that fits into automated pipelines.

Section B — Scenario-Based Descriptive Questions

Q5 [Scenario-Based]

Scenario: You are a junior analyst at a healthcare company. Your manager asks you to use AI to analyse patient readmission rates. The AI keeps giving generic statistical responses that do not align with your clinical database structure or the team's goals. What specific Analytical Persona prompt would you design? Write out the full persona prompt and explain each component you included.

Answer

This scenario is a classic case where a persona is missing — the AI doesn't know it's working in healthcare, doesn't know the tools, and doesn't know the goal.

My designed persona prompt:

"You are a Senior Clinical Data Analyst with 8 years of experience in hospital operations and patient outcomes analytics.

You work at a large multi-specialty hospital that uses SQL Server and Python (pandas, matplotlib) for analysis.

Your goal is to help the clinical team reduce 30-day patient readmission rates by identifying high-risk patient segments.

You always reference ICD-10 diagnosis codes correctly, validate date ranges for admission and discharge, and explain your SQL logic.

You never use SELECT * and always filter for completed, non-test patient records.

Now answer the following question: [ACTUAL QUESTION]"

Why each component matters:

- Role + Experience: Anchors the AI in clinical analytics, not generic stats
- Tools: Ensures SQL is written for SQL Server syntax, not MySQL or PostgreSQL
- Goal: The AI now prioritises readmission-relevant metrics (e.g., LOS, diagnosis codes)
- Rules: Prevents ICD code hallucinations and ensures data hygiene
- Restrictions: Avoids lazy SELECT * queries that could expose PHI unnecessarily

Q6 [Scenario-Based]

Scenario: You ask an AI to calculate the compound annual growth rate (CAGR) for your company's revenue over 5 years. The AI immediately gives you a final number — 14.2% — with no explanation. Your finance manager says the number seems off. How would you redesign the prompt using Chain-of-Thought techniques to get a verifiable, step-by-step answer? Write the improved prompt and explain why each addition helps.

Answer

The original prompt failed because it let the AI skip directly to a result — which LLMs often get wrong for multi-step math. The fix is Structured CoT.

Improved prompt:

"Calculate the Compound Annual Growth Rate (CAGR) for the following revenue data:

Year 0 (Base): $2,400,000

Year 5 (End): $4,650,000

Number of Years: 5

Show your answer by breaking it into clear steps in this exact format:

Step 1: Identify the CAGR formula

Step 2: Substitute the values into the formula

Step 3: Calculate the ratio (End Value / Beginning Value)

Step 4: Raise to the power of (1/n)

Step 5: Subtract 1 and convert to percentage

Final Answer: CAGR = X%"

Why each addition helps:

- Explicit values: Eliminates ambiguity — AI cannot misinterpret the inputs
- Forced format: Each step is labelled; your manager can verify Step 2 without trusting the final answer blindly

• Formula first: Asking for the formula in Step 1 checks whether the AI even knows the correct formula before it calculates

• Auditable: If the finance manager disagrees at Step 3, you know exactly where to challenge the AI — not just the final number

Q7 [Scenario-Based]

Scenario: You are onboarding a new analyst to your e-commerce team. The team uses a complex PostgreSQL database with 12 tables. The new analyst wants to use AI to help write queries, but previous AI attempts produced SQL with columns that don't exist. Design a Few-Shot Learning prompt template for the new analyst to use. Include schema injection, two Gold Standard examples, and explain what makes each example "gold standard."

Answer

This is exactly the use case Few-Shot Learning was built for. Here is the template I would design:

--- SCHEMA ---

Table: orders (order_id INT, customer_id INT, order_date DATE, revenue FLOAT, status VARCHAR)

Table: customers (customer_id INT, full_name VARCHAR, region VARCHAR, signup_date DATE)

Table: products (product_id INT, product_name VARCHAR, category VARCHAR, unit_price FLOAT)

--- GOLD STANDARD EXAMPLE 1: Monthly Revenue ---

Question: Find total revenue per month for completed orders.

Query:

```
SELECT DATE_TRUNC('month', order_date) AS month,
SUM(revenue) AS total_revenue
FROM orders
WHERE status = 'completed'
GROUP BY 1 ORDER BY 1;
```

Why it is Gold Standard: Uses real column names, filters correctly on status, uses DATE_TRUNC for PostgreSQL date handling, clean GROUP BY alias.

--- GOLD STANDARD EXAMPLE 2: Regional Customer Count ---

Question: Count distinct customers per region.

Query:

```
SELECT region, COUNT(DISTINCT customer_id) AS customer_count
```

FROM customers
GROUP BY region ORDER BY customer_count DESC;

Why it is Gold Standard: Uses DISTINCT correctly, proper alias, clean ORDER BY — no SELECT *, no invented columns.

--- NEW QUESTION ---
Now write a query to: [NEW ANALYST's QUESTION]

Key insight for the new analyst: The AI will now follow these exact patterns — using real column names, the right syntax, and the team's style conventions.

Q8 [Scenario-Based]

Scenario: You run an AI-generated SQL query in Redshift and receive this error: "column "sale_amount" does not exist." You also notice the result should show only Q3 2024 data, but no date filter was applied. Using the Iterative Debugging framework, write out your complete Round 2 correction prompt. Explain which debugging sub-technique you are applying at each step.

Answer

This scenario has two problems: a hallucinated column name AND a missing date filter. Iterative Debugging handles both without restarting.

My Round 2 Correction Prompt:
"The query you wrote contains two errors. Please fix both and return the corrected full query.

Error 1 — Hallucinated Column Name [Schema Reinforcement]:
You used "sale_amount" but that column does not exist.
The correct column is "revenue" in the orders table.

Here is the actual schema to reference:
Table: orders (order_id INT, customer_id INT, order_date DATE, revenue FLOAT, status VARCHAR)

Error 2 — Missing Date Filter [Targeted Correction]:
The query should only include orders from Q3 2024 (July 1 to September 30, 2024).
Add a WHERE clause: order_date BETWEEN '2024-07-01' AND '2024-09-30'

Additional Constraint [Constraint Adding]:
Also filter for status = 'completed' only.

Please fix ONLY these two issues and re-output the complete corrected query."

Sub-techniques applied:

- Schema Reinforcement — Re-injecting real column names to fix hallucination
- Targeted Correction — Pointing to the missing date filter specifically, not rewriting everything
- Constraint Adding — Layering in the status filter as a new rule during the debug conversation
- Error Explanation — Pasting the actual DB error message so the AI can self-diagnose precisely

Section C — Coding Questions

Q9 [Coding]

Write an Analytical Persona prompt in Python that uses the OpenAI / Anthropic API to query a data science assistant. The persona should be a Senior Data Scientist working in fintech, restricted from using SELECT *, and asked to find the top 5 customers by transaction volume in the last 30 days.

Answer

Here I'm constructing the persona as a system message and the actual query as a user message — which is the standard pattern for AI APIs.

The key design choices:

- System message = persona definition (role, context, rules)
- User message = the actual analytical question
- Separating these ensures the persona "sticks" across multi-turn conversations

Code:

```
import openai

client = openai.OpenAI(api_key="YOUR_API_KEY")

system_persona = """
```

```
You are a Senior Data Scientist with 10 years of experience
in fintech and payment analytics.
You work at a digital payments company using PostgreSQL and dbt.
Your goal is to help the risk and growth teams analyse
customer transaction behaviour.
You always write clean, well-commented SQL with proper aliases.
You never use SELECT * and always validate data types.
You always include a WHERE clause to filter relevant date ranges.
"""

user_question = """
Write a SQL query to find the top 5 customers
by total transaction volume in the last 30 days.
Exclude declined and refunded transactions.
"""

response = client.chat.completions.create(
 model="gpt-4",
 messages=[
 {"role": "system", "content": system_persona},
 {"role": "user", "content": user_question}
 ]
)

print(response.choices[0].message.content)
```

Q10 [Coding]

Write a Python function called cot_prompt_builder() that accepts a math question and automatically wraps it with a Structured Chain-of-Thought template. The function should return the formatted prompt string, and include a sample call demonstrating its use for a Month-over-Month growth rate calculation.

Answer

This function is useful when you have many analytical questions that all need step-by-step reasoning — you don't want to manually write the CoT template every time.

The function parametrises the CoT structure so it can be reused across different question types (revenue growth, churn rate, conversion %, etc.).

Code:

```
def cot_prompt_builder(question: str, steps: list[str] = None) -> str:
 """
```

```
    Wraps a question with a Structured Chain-of-Thought template.
    Args:
    question: The analytical question to answer.
    steps: Optional list of step labels for structured reasoning.
    Returns:
    Formatted CoT prompt string.
    """
    default_steps = [
    "Identify all given values and the formula required.",
    "Substitute the values into the formula.",
    "Perform the calculation step by step.",
    "Convert to the required unit or percentage.",
    "State the Final Answer clearly."
    ]
    steps = steps or default_steps

    step_instructions = "\n".join(
    f" Step {i+1}: {s}" for i, s in enumerate(steps)
    )

    prompt = f"""
Answer the following question by showing each reasoning step clearly.
Format your answer EXACTLY as shown below:

{step_instructions}
 Final Answer: [Your result with units]

Question: {question}
"""
    return prompt.strip()



# ── Sample Call ──────────────────────────────────────────
question = (
 "What is the Month-over-Month revenue growth rate "
 "if January revenue was $120,000 and "
 "February revenue was $145,000?"
)

formatted_prompt = cot_prompt_builder(question)
print(formatted_prompt)
```

Q11 [Coding]

Write a Python function called few_shot_prompt_builder() that accepts: (1) a schema dictionary, (2) a list of (question, SQL) tuples as examples, and (3) a new question. It should return a complete Few-Shot prompt string ready to send to an AI model.

Answer

Few-Shot prompts have a consistent structure: schema first, then examples, then the new question. This function makes that reusable — so any analyst on the team can produce a well-formed prompt without manually writing boilerplate.

Code:

```
def few_shot_prompt_builder(
 schema: dict,
 examples: list[tuple[str, str]],
 new_question: str
) -> str:
 """
 Builds a Few-Shot Learning prompt with schema injection.
 Args:
 schema: {table_name: [column_definitions]}
 examples: [(question_str, sql_str), ...]
 new_question: The new question for the AI to answer.
 Returns:
 Complete few-shot prompt string.
 """
 # ── Schema Section ──────────────────────────────────
 schema_lines = ["Here is the database schema:\n"]
 for table, cols in schema.items():
 col_str = ", ".join(cols)
 schema_lines.append(f" Table: {table} ({col_str})")

 # ── Examples Section ────────────────────────────────
 example_lines = ["\nHere are Gold Standard example queries:\n"]
 for i, (q, sql) in enumerate(examples, 1):
 example_lines.append(f" Example {i} — {q}")
 example_lines.append(f" SQL:\n {sql}\n")

 # ── New Question ────────────────────────────────────
 question_line = f"\nNow write a SQL query to:\n {new_question}"

 return "\n".join(schema_lines + example_lines) + question_line


# ── Sample Call ──────────────────────────────────────
schema = {
 "orders": ["order_id INT", "customer_id INT",
```

```
    "order_date DATE", "revenue FLOAT", "status VARCHAR"],
    "customers": ["customer_id INT", "full_name VARCHAR",
    "region VARCHAR", "signup_date DATE"]
}

examples = [
    ("Find monthly revenue",
    "SELECT DATE_TRUNC('month', order_date) AS month,\n"
    " SUM(revenue) AS total_revenue\n"
    "FROM orders WHERE status = 'completed'\n"
    "GROUP BY 1 ORDER BY 1;")
]

prompt = few_shot_prompt_builder(
    schema, examples,
    "Find the top 5 regions by revenue in Q1 2024."
)
print(prompt)
```

Q12 [Coding]

Write a Python function called iterative_debug_prompt() that takes: (1) the original broken query, (2) the database error message, (3) a correction dictionary {wrong_name: correct_name}, and (4) any additional constraints. It should return a structured Round 2 correction prompt for the AI.

Answer

This function automates the most common iterative debugging pattern — schema mismatch plus DB error message. It's reusable across any debugging session without re-typing the boilerplate correction structure.

Code:

```
def iterative_debug_prompt(
    broken_query: str,
    error_message: str,
    corrections: dict,
    schema: dict = None,
    extra_constraints: list[str] = None
) -> str:
    """
    Builds an Iterative Debugging Round 2 correction prompt.
    Args:
    broken_query: The original SQL query that failed.
    error_message: The actual DB error returned.
```

```
    corrections: {wrong_name: correct_name} for column fixes.
    schema: Optional schema dict to re-inject.
    extra_constraints: Optional list of new rules to add.
    Returns:
    A structured correction prompt string.
    """
    lines = [
    "The query you wrote has errors. Please fix them and return",
    "the complete corrected query.\n",
    "--- ORIGINAL BROKEN QUERY ---",
    broken_query,
    "",
    "--- DATABASE ERROR MESSAGE ---",
    error_message,
    "",
    "--- CORRECTIONS NEEDED ---",
    ]

    for wrong, correct in corrections.items():
    lines.append(
    f" '{wrong}' does not exist. Use '{correct}' instead."
    )

    if schema:
    lines.append("\n--- ACTUAL SCHEMA (use ONLY these names) ---")
    for table, cols in schema.items():
    lines.append(f" Table: {table} ({', '.join(cols)})")

    if extra_constraints:
    lines.append("\n--- ADDITIONAL CONSTRAINTS ---")
    for c in extra_constraints:
    lines.append(f" • {c}")

    lines.append("\nPlease fix ONLY the issues above and output the full
corrected query.")
    return "\n".join(lines)



# ── Sample Call ──────────────────────────────────────────
prompt = iterative_debug_prompt(
    broken_query="""SELECT client_name, SUM(sale_amount)
FROM sales_data GROUP BY client_name;""",
    error_message="ERROR: column \"client_name\" does not exist",
    corrections={
```

```
    "client_name": "full_name (in customers table)",
    "sale_amount": "revenue (in orders table)",
    "sales_data": "orders (table name)"
    },
    schema={
    "orders": ["order_id INT", "customer_id INT",
    "revenue FLOAT", "status VARCHAR"],
    "customers": ["customer_id INT", "full_name VARCHAR", "region VARCHAR"]
    },
    extra_constraints=["Filter status = 'completed' only"]
)
print(prompt)
```

Section D — Database Scenario-Based Coding (SQL)

Q13 [DB Scenario + SQL]

Scenario: You are a data analyst at a SaaS company. Using the schema below, write an AI-ready Few-Shot prompt that includes the schema, one Gold Standard example, and then solve the new question yourself with a complete SQL query. Schema: Table: subscriptions (sub_id INT, customer_id INT, plan VARCHAR, start_date DATE, end_date DATE, monthly_fee FLOAT, status VARCHAR) Table: customers (customer_id INT, full_name VARCHAR, country VARCHAR, signup_date DATE) New Question: Find the total Monthly Recurring Revenue (MRR) by plan type for all active subscriptions.

Answer

I'll structure this as both the Few-Shot prompt (what you'd send to the AI) AND the direct SQL solution — since the question asks for both.

The Few-Shot prompt to send to AI:

Schema: subscriptions + customers (as above)

Gold Standard Example: Count active subscriptions per country

Direct SQL Solution:

Code:

```
-- Few-Shot Gold Standard Example (included in prompt):
-- Q: Count active subscriptions per country.
SELECT c.country,
 COUNT(s.sub_id) AS active_subscriptions
```

```
FROM subscriptions s
JOIN customers c ON s.customer_id = c.customer_id
WHERE s.status = 'active'
GROUP BY c.country
ORDER BY active_subscriptions DESC;



-- ── SOLUTION: MRR by Plan Type ──────────────────────────
SELECT
 plan,
 COUNT(sub_id) AS active_subscriptions,
 SUM(monthly_fee) AS total_mrr,
 ROUND(AVG(monthly_fee), 2) AS avg_fee_per_plan
FROM subscriptions
WHERE status = 'active'
GROUP BY plan
ORDER BY total_mrr DESC;

-- Why this is correct:
-- • Filters status = 'active' to exclude churned/cancelled subs
-- • SUM(monthly_fee) gives true MRR per plan
-- • AVG gives insight into plan pricing distribution
-- • ORDER BY total_mrr DESC shows highest-revenue plans first
```

Q14 [DB Scenario + SQL]

Scenario: You are debugging an AI-generated query at a retail company. The AI produced the following broken query: SELECT p.product_name, SUM(o.qty) AS total_sold FROM order_items o JOIN product_catalog p ON o.prod_id = p.product_id WHERE o.sale_date >= CURRENT_DATE - 90 GROUP BY p.product_name ORDER BY total_sold DESC LIMIT 10; But the actual schema is: Table: order_items (item_id INT, order_id INT, product_id INT, quantity INT, item_date DATE) Table: products (product_id INT, name VARCHAR, category VARCHAR, price FLOAT) Identify ALL hallucinations in the query and write: (a) the correction prompt, (b) the fully corrected SQL.

Answer

Let me identify every hallucination systematically before writing the fix:

Hallucinations found:

1. "order_items.qty" → correct column is "quantity"
2. "order_items.prod_id" → correct column is "product_id"
3. "product_catalog" → correct table is "products"

4. "product_catalog.product_name" → correct column is "products.name"
5. "o.sale_date" → correct column is "item_date"

(a) Correction Prompt to send to AI:

"Your query has 5 schema errors. Here is the real schema:
Table: order_items (item_id, order_id, product_id, quantity, item_date)
Table: products (product_id, name, category, price)
Fixes: qty→quantity, prod_id→product_id, product_catalog→products,
product_name→name, sale_date→item_date.
Rewrite using ONLY these exact names."

Code:

```
-- (b) Fully Corrected SQL:
SELECT
 p.name AS product_name,
 SUM(oi.quantity) AS total_sold,
 COUNT(DISTINCT oi.order_id) AS order_count
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
WHERE oi.item_date >= CURRENT_DATE - INTERVAL '90 days'
GROUP BY p.name
ORDER BY total_sold DESC
LIMIT 10;

-- Changes made:
-- qty → quantity
-- prod_id → product_id (JOIN condition fixed)
-- product_catalog → products (correct table name)
-- product_name → name (correct column in products)
-- sale_date → item_date (correct date column)
-- Added INTERVAL syntax for standard PostgreSQL compatibility
```


Q15 [DB Scenario + SQL]

Scenario: You are a data engineer at an e-commerce company. Your manager wants a weekly report showing: (1) total revenue by region, (2) average order value by region, and (3) the percentage each region contributes to overall revenue — all for completed orders in the current year. Using the Delimiters & Structured Output technique, write: (a) the full prompt with delimiters requesting JSON output, and (b) the equivalent SQL query that produces this data.

Answer

(a) Full Prompt with Delimiters (requesting JSON):

###SCHEMA###
Table: orders (order_id INT, customer_id INT, order_date DATE, revenue FLOAT, status VARCHAR)
Table: customers (customer_id INT, full_name VARCHAR, region VARCHAR)

###TASK###
You are a Senior Data Analyst. Generate a SQL query and its expected JSON output structure.
The report must show for completed orders in 2024:
- Total revenue by region
- Average order value by region
- Each region's % share of overall revenue

###OUTPUT FORMAT###
Return your answer ONLY as a JSON array with this structure:
[{"region": "...", "total_revenue": 0, "avg_order_value": 0, "revenue_share_pct": 0}]
No explanation text. No markdown. Pure JSON only.

Code:

```
-- (b) SQL Query — Regional Revenue Report with % Share:
WITH regional_revenue AS (
 SELECT
 c.region,
 COUNT(o.order_id) AS order_count,
 SUM(o.revenue) AS total_revenue,
 ROUND(AVG(o.revenue), 2) AS avg_order_value
 FROM orders o
 JOIN customers c ON o.customer_id = c.customer_id
 WHERE o.status = 'completed'
 AND EXTRACT(YEAR FROM o.order_date) = EXTRACT(YEAR FROM CURRENT_DATE)
 GROUP BY c.region
),
overall AS (
 SELECT SUM(total_revenue) AS grand_total
 FROM regional_revenue
)
SELECT
 r.region,
 r.order_count,
 r.total_revenue,
 r.avg_order_value,
```

```
 ROUND(
 (r.total_revenue / o.grand_total) * 100, 2
 ) AS revenue_share_pct
FROM regional_revenue r
CROSS JOIN overall o
ORDER BY r.total_revenue DESC;

-- Pattern handbook:
-- CTE 1: aggregates per region (total, count, avg)
-- CTE 2: grand total for % share calculation
-- CROSS JOIN: attaches grand_total to every region row
-- ROUND(..., 2): ensures clean JSON-serialisable decimals
```

📋 Quick Reference — Techniques at a Glance

#	Technique	Core Idea	When to Use
1.1	Analytical Persona	Define expert role before asking	Every analytics prompt
1.2	Chain-of-Thought	Show step-by-step reasoning	Math & multi-step queries
1.3	Few-Shot Learning	Provide schema + Gold Standard queries	Complex or new schema work
1.4	Delimiters & Output	Separate sections, specify format	Automation & pipelines
1.5	Iterative Debugging	Feed errors back, correct incrementally	When AI output has errors